



A Case Study on
Database Design for a Library Management System
Resource Tracking, Lending Operations, and Fine Administration

A9511 - Database Management Systems

Submitted by
Batch No. 6

Madhavarapu Saritha	25881A05V7
Chepyala Vishal	25881A05X9
Gundu Srijay Krishna	25881A05X0

Course Facilitator
Dr. Reddy Saisindhutheja
Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

November 2026

Contents

1	Problem Definition and Context	2
1.1	Library Management as a Database Problem	2
1.2	Relevance to Sustainable Development Goals (SDGs)	2
2	Problem Analysis and Solution Development	2
2.1	Characterization of the Problem	2
2.2	Engineering Knowledge Applied	3
2.3	Title vs. Physical Copy and How Members, Issues and Returns Relate . .	3
2.4	Issue Rules and Fine Calculation Logic	6
2.5	Handling of Reservations When All Copies Are Issued	8
2.6	Overdue, Popular-Title and Member-wise Fine Queries	11
2.7	Testing and Validation	12
2.8	Proposed Solution Strategy	13
3	Sustainability Integration	14
3.1	Resource Efficiency and Data-Driven Acquisition	14
3.2	Impact on Society and Environment	14
4	Conclusion and Future Directions	14
4.1	Solution Summary	14
4.2	Future Scope	14

1 Problem Definition and Context

1.1 Library Management as a Database Problem

A college library lends thousands of books every semester to students, faculty and staff. Many libraries still keep this information in paper registers or loosely structured spreadsheets. As a result, it is difficult to answer simple questions such as “*Is a copy of this book on the shelf?*”, “*Who has had this copy for more than two weeks?*” or “*How much does this member owe?*”. Errors in these records lead to lost books, disputes over fines and unfair treatment of members waiting for popular titles.

Problem Statement: Model books, copies, members, issue/return records, fines and reservations. Build the schema, stored procedures for issuing and returning books, and queries for overdue books, popular titles and member-wise fines.

The case study analyses four questions:

- **Title vs. copy:** the difference between a book title and its physical copies, and how members, issues and returns relate.
- **Issue rules:** borrowing limits, due dates and the fine calculation logic.
- **Reservations:** how requests are handled when all copies of a title are issued.
- **Reporting:** overdue, popular-title and member-wise fine queries.

The solution is implemented and tested on **PostgreSQL 16** using PL/pgSQL stored procedures. Every output in this report was produced by actually running the scripts on the sample data described in Section 2.7.

1.2 Relevance to Sustainable Development Goals (SDGs)

A well-designed library database supports the following UN SDGs:

- **SDG 4 (Quality Education):** Reliable, quick access to textbooks and reference material supports Target 4.3 (equal access to quality tertiary education).
- **SDG 10 (Reduced Inequalities):** A first-come-first-served reservation queue and capped fines make sure students who cannot afford their own textbooks get fair access to shared copies.
- **SDG 12 (Responsible Consumption and Production):** Sharing and reusing physical copies, and buying new copies only where demand data justifies it, reduces paper and resource waste (Target 12.5).

2 Problem Analysis and Solution Development

2.1 Characterization of the Problem

At first glance a library looks like a simple “books and borrowers” application. In practice, however, the database has to enforce several rules that interact with each other:

- A *title* (e.g. *Database System Concepts*, 7th ed.) has many *physical copies*. Members borrow copies, but they search for and reserve titles.
- Borrowing limits, loan periods and fine rates depend on the member category (student, faculty, staff).

- A copy that is returned while other members are waiting must go to the *right* person in the queue, not back to the open shelf.
- Two librarians working at two counters must never lend the same copy twice, or let a member exceed their limit by issuing at the same moment.

These rules belong *inside* the database, as constraints and stored procedures, so that no front-end program can bypass them.

2.2 Engineering Knowledge Applied

- **Conceptual Modelling:** Entity–Relationship (ER) modelling with cardinality and participation constraints.
- **Normalization:** Functional-dependency analysis to bring every relation to Third Normal Form (3NF)/BCNF.
- **Integrity Constraints:** Primary/foreign keys, **CHECK** constraints and *partial unique indexes* that encode business rules declaratively.
- **Procedural SQL:** PL/pgSQL stored procedures for issue, return, reservation, hold expiry and fine payment.
- **Transactions and Concurrency Control:** Row-level locks (**SELECT** ... **FOR UPDATE**) and a fixed lock order to prevent double issue and deadlocks.
- **Analytical SQL:** Joins, aggregation with **FILTER**, Common Table Expressions (CTEs) and window functions (**RANK()**).

2.3 Title vs. Physical Copy and How Members, Issues and Returns Relate

The central design decision is to keep the **bibliographic title** (book) separate from the **physical item** (book_copy):

- **book** holds what is common to every copy: ISBN, title, edition, publisher, year. A title is described once, no matter how many copies the library buys.
- **book_copy** holds what differs between copies: a unique barcode, shelf location, acquisition date and the *current status* (**AVAILABLE**, **ISSUED**, **ON_HOLD**, **LOST**, **DAMAGED**).
- A **loan** links one *copy* to one *member* for a period of time. The issue is the **INSERT**; the return is the **UPDATE** that fills **return_date**. One row therefore records the whole issue/return cycle.
- A **reservation** is placed on a *title*, not on a copy: a member wants “any copy of *Clean Code*”, whichever is returned first.
- A **fine** is generated by at most one loan, so **fine.loan_id** is **UNIQUE**.

Figure 1 shows the resulting relational schema. Arrows point from a foreign key to the referenced (parent) table.

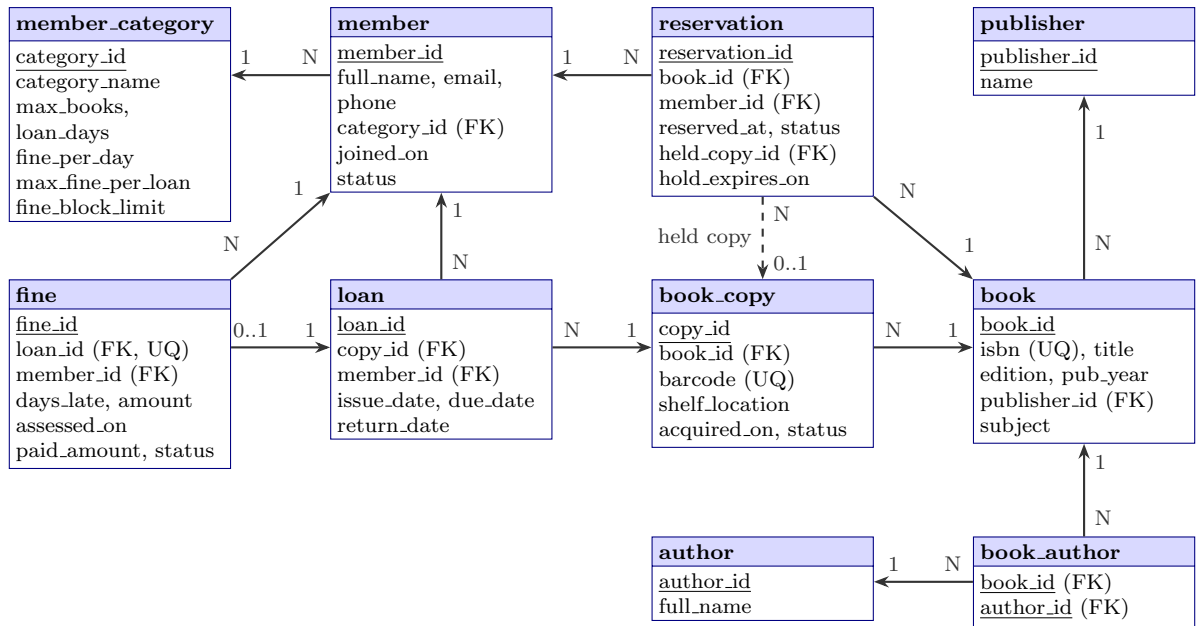


Figure 1: Relational schema of the Library Management System (PK underlined, FK = foreign key, UQ = unique)

2.3.1 Normalization

The main functional dependencies (FDs) are:

$\text{isbn} \rightarrow \text{title, edition, publisher_id, pub_year, subject}$
 $\text{barcode} \rightarrow \text{copy_id, book_id, shelf_location, acquired_on, status}$
 $\text{category_id} \rightarrow \text{max_books, loan_days, fine_per_day, ...}$
 $\text{loan_id} \rightarrow \text{copy_id, member_id, issue_date, due_date, return_date}$

If title data were stored in every copy row, the FD $\text{isbn} \rightarrow \text{title}$ would be a transitive dependency ($\text{copy_id} \rightarrow \text{isbn} \rightarrow \text{title}$), which violates 3NF and causes update anomalies (changing a title in one copy but not another). Likewise, storing max_books in **member** would repeat the rule for every student. Splitting into **book/book_copy** and **member/member_category** leaves every non-key attribute depending only on its table's key. Authors are many-to-many with books, so they are resolved through **book_author**. Every relation is therefore in **BCNF**.

2.3.2 Schema Highlights

Listing 1 shows the core tables. Two partial unique indexes encode business rules that a plain UNIQUE constraint cannot express:

- **uq_loan_open_copy**: a copy can be on *at most one open loan* (one with **return_date** IS NULL), while any number of closed loans are allowed as history.
- **uq_resv_live**: a member may hold only one *live* (WAITING/READY) reservation per title.

Listing 1: Core tables and constraints (from 01_schema.sql)

```

1 CREATE TABLE book (
2     book_id    SERIAL          PRIMARY KEY,
3     isbn       CHAR(13)       NOT NULL UNIQUE,
4     title      VARCHAR(200)   NOT NULL,
5     edition    SMALLINT       NOT NULL DEFAULT 1,
6     publisher_id INT          REFERENCES publisher,

```

```

7      pub_year    SMALLINT    CHECK (pub_year BETWEEN 1800 AND 2100),
8      subject     VARCHAR(60)
9  );
10 CREATE TABLE book_copy (
11      copy_id     SERIAL      PRIMARY KEY,
12      book_id     INT         NOT NULL REFERENCES book,
13      barcode     VARCHAR(20) NOT NULL UNIQUE,
14      shelf_location VARCHAR(20),
15      acquired_on DATE        NOT NULL DEFAULT CURRENT_DATE,
16      status      VARCHAR(10) NOT NULL DEFAULT 'AVAILABLE'
17      CHECK (status IN ('AVAILABLE', 'ISSUED', 'ON_HOLD', 'LOST', 'DAMAGED'))
18 );
19 CREATE TABLE loan (
20      loan_id     SERIAL      PRIMARY KEY,
21      copy_id     INT         NOT NULL REFERENCES book_copy,
22      member_id   INT         NOT NULL REFERENCES member,
23      issue_date  DATE        NOT NULL DEFAULT CURRENT_DATE,
24      due_date    DATE        NOT NULL,
25      return_date DATE,
26      CHECK (due_date > issue_date),
27      CHECK (return_date IS NULL OR return_date >= issue_date)
28 );
29 -- A copy can be on at most ONE open loan at any time
30 CREATE UNIQUE INDEX uq_loan_open_copy ON loan (copy_id) WHERE return_date IS NULL;
31
32 CREATE TABLE reservation (
33      reservation_id SERIAL      PRIMARY KEY,
34      book_id     INT         NOT NULL REFERENCES book,
35      member_id   INT         NOT NULL REFERENCES member,
36      reserved_at  TIMESTAMP    NOT NULL DEFAULT now(),
37      status      VARCHAR(10)  NOT NULL DEFAULT 'WAITING'
38      CHECK (status IN ('WAITING', 'READY', 'FULFILLED', 'CANCELLED', 'EXPIRED')),
39      held_copy_id INT         REFERENCES book_copy,
40      hold_expires_on DATE,
41      CHECK (status <> 'READY' OR (held_copy_id IS NOT NULL AND hold_expires_on IS NOT NULL)),
42      CHECK (status <> 'WAITING' OR held_copy_id IS NULL)
43 );
44 CREATE UNIQUE INDEX uq_resv_live ON reservation (book_id, member_id)
45 WHERE status IN ('WAITING', 'READY');
46
47 CREATE TABLE fine (
48      fine_id     SERIAL      PRIMARY KEY,
49      loan_id     INT         NOT NULL UNIQUE REFERENCES loan,
50      member_id   INT         NOT NULL REFERENCES member,
51      days_late   INT         NOT NULL CHECK (days_late > 0),
52      amount      NUMERIC(8,2) NOT NULL CHECK (amount >= 0),
53      assessed_on DATE        NOT NULL DEFAULT CURRENT_DATE,
54      paid_amount NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (paid_amount >= 0),
55      status      VARCHAR(8)  NOT NULL DEFAULT 'UNPAID'
56      CHECK (status IN ('UNPAID', 'PAID', 'WAIVED')),
57      CHECK (paid_amount <= amount)
58 );

```

A view `v_title_availability` combines both levels for the circulation desk. Table 1 shows its output for the sample data: the library holds 8 titles but 13 physical copies.

Table 1: Title-level availability computed from copy-level status (before the test run)

Title	Copies	Available	Issued	On hold	Queue
Database System Concepts	3	1	2	0	0
Fundamentals of Database Systems	2	1	1	0	0
Introduction to Algorithms	2	1	1	0	0
Operating System Concepts	2	1	1	0	0
Computer Networks	1	0	1	0	1
Clean Code	1	0	1	0	2
The Pragmatic Programmer	1	1	0	0	0
Design Patterns	1	1	0	0	0

2.4 Issue Rules and Fine Calculation Logic

2.4.1 Borrowing Rules

Rules are stored as *data* in `member_category`, not hard-coded, so the library committee can change them without touching any program (Table 2).

Table 2: Borrowing rules per member category

Category	Max books	Loan days	Fine/day	Cap per loan	Block if unpaid >
Student	3	14	Rs. 2	Rs. 200	Rs. 100
Faculty	8	30	Rs. 1	Rs. 100	Rs. 200
Staff	4	21	Rs. 1	Rs. 100	Rs. 100

2.4.2 Mathematical Formulation

Let member m belong to category c with borrowing limit L_c , loan period T_c , daily fine rate r_c , fine cap $F_c^{\max}$ and blocking limit B_c .

Eligibility to borrow. With n_m = number of open loans, o_m = number of overdue open loans and U_m = total unpaid fines,

$$\text{Eligible}(m) \iff s_m = \text{ACTIVE} \wedge o_m = 0 \wedge n_m < L_c \wedge U_m \leq B_c$$

Due date. For a copy issued on date d_i :

$$d_{\text{due}} = d_i + T_c$$

Fine. When the copy is returned on date d_r :

$$F = \min \left(r_c \cdot \max(0, d_r - d_{\text{due}}), F_c^{\max} \right)$$

Worked example: a student returns a book 26 days late, so $F = \min(2 \times 26, 200) = \text{Rs. } 52$. If the same book were 120 days late, $F = \min(240, 200) = \text{Rs. } 200$. The cap keeps a forgotten book from turning into an unaffordable debt.

Expected wait for a reservation. If a title has n copies, all issued, and a member is at position k in the queue, then in the worst case the member waits for

$$W_k \leq \left\lceil \frac{k}{n} \right\rceil \cdot T_c + \left(\left\lceil \frac{k}{n} \right\rceil - 1 \right) \cdot H$$

days, where $H = 3$ days is the pick-up window given to each earlier member in the queue.

2.4.3 Fine Function and Issue Procedure

Listing 2: Fine calculation and the `sp_issue_book` procedure

```

1 CREATE OR REPLACE FUNCTION fn_calculate_fine(p_due DATE, p_returned DATE,
2                                           p_rate NUMERIC, p_cap NUMERIC)
3 RETURNS NUMERIC LANGUAGE sql IMMUTABLE AS $$
4     SELECT LEAST(GREATEST(p_returned - p_due, 0) * p_rate, p_cap)::NUMERIC(8,2);
5 $$;
6
7 CREATE OR REPLACE PROCEDURE sp_issue_book(p_member_id INT, p_barcode VARCHAR,
8                                           INOUT p_loan_id INT DEFAULT NULL,
9                                           INOUT p_due_date DATE DEFAULT NULL)
10 LANGUAGE plpgsql AS $$
11 DECLARE
12     v_mem      RECORD;
13     v_copy     book_copy%ROWTYPE;

```

```

14     v_active    INT;
15     v_overdue   INT;
16     v_unpaid    NUMERIC;
17 BEGIN
18     -- 1. Lock the member: two parallel issues cannot both pass the limit check
19     SELECT m.status, c.max_books, c.loan_days, c.fine_block_limit INTO v_mem
20         FROM member m JOIN member_category c USING (category_id)
21         WHERE m.member_id = p_member_id
22         FOR UPDATE OF m;
23     IF NOT FOUND THEN
24         RAISE EXCEPTION 'Member % does not exist', p_member_id;
25     END IF;
26     IF v_mem.status <> 'ACTIVE' THEN
27         RAISE EXCEPTION 'Member % is %', p_member_id, v_mem.status;
28     END IF;
29
30     -- 2. Borrowing rules
31     SELECT COUNT(*), COUNT(*) FILTER (WHERE due_date < CURRENT_DATE)
32         INTO v_active, v_overdue
33         FROM loan
34         WHERE member_id = p_member_id AND return_date IS NULL;
35
36     IF v_overdue > 0 THEN
37         RAISE EXCEPTION 'Member % has % overdue book(s); return them first',
38             p_member_id, v_overdue;
39     END IF;
40     IF v_active >= v_mem.max_books THEN
41         RAISE EXCEPTION 'Borrowing limit reached: % of % books already issued',
42             v_active, v_mem.max_books;
43     END IF;
44
45     SELECT COALESCE(SUM(amount - paid_amount), 0) INTO v_unpaid
46         FROM fine WHERE member_id = p_member_id AND status = 'UNPAID';
47     IF v_unpaid > v_mem.fine_block_limit THEN
48         RAISE EXCEPTION 'Unpaid fines Rs.% exceed the limit of Rs.%',
49             v_unpaid, v_mem.fine_block_limit;
50     END IF;
51
52     -- 3. Lock the copy and make sure it may be lent to THIS member
53     SELECT * INTO v_copy FROM book_copy WHERE barcode = p_barcode FOR UPDATE;
54     IF NOT FOUND THEN
55         RAISE EXCEPTION 'No copy with barcode %', p_barcode;
56     END IF;
57
58     IF v_copy.status = 'ON_HOLD' THEN
59         UPDATE reservation SET status = 'FULFILLED'
60             WHERE held_copy_id = v_copy.copy_id AND status = 'READY'
61             AND member_id = p_member_id;
62         IF NOT FOUND THEN
63             RAISE EXCEPTION 'Copy % is on hold for another member', p_barcode;
64         END IF;
65     ELSIF v_copy.status <> 'AVAILABLE' THEN
66         RAISE EXCEPTION 'Copy % is not available (status %)', p_barcode, v_copy.status;
67     END IF;
68
69     IF EXISTS (SELECT 1 FROM loan l JOIN book_copy c USING (copy_id)
70         WHERE l.member_id = p_member_id AND l.return_date IS NULL
71         AND c.book_id = v_copy.book_id) THEN
72         RAISE EXCEPTION 'Member % already has a copy of this title', p_member_id;
73     END IF;
74
75     -- 4. Record the loan
76     p_due_date := CURRENT_DATE + v_mem.loan_days;
77     INSERT INTO loan (copy_id, member_id, issue_date, due_date)
78     VALUES (v_copy.copy_id, p_member_id, CURRENT_DATE, p_due_date)
79     RETURNING loan_id INTO p_loan_id;
80
81     UPDATE book_copy SET status = 'ISSUED' WHERE copy_id = v_copy.copy_id;
82 END $$;

```

2.4.4 Return Procedure

Returning a copy closes the loan, assesses the fine with the formula above and then hands the copy to the reservation queue (Section 2.5).

Listing 3: The `sp_return_book` procedure

```
1 CREATE OR REPLACE PROCEDURE sp_return_book(p_barcode VARCHAR,
2                                           p_return_date DATE DEFAULT CURRENT_DATE,
3                                           INOUT p_fine NUMERIC DEFAULT 0,
4                                           INOUT p_held_for INT DEFAULT NULL)
5 LANGUAGE plpgsql AS $$
6 DECLARE
7     v_loan loan%ROWTYPE;
8     v_rate NUMERIC;
9     v_cap NUMERIC;
10 BEGIN
11     SELECT l.* INTO v_loan
12     FROM loan l JOIN book_copy c USING (copy_id)
13     WHERE c.barcode = p_barcode AND l.return_date IS NULL
14     FOR UPDATE OF l, c;
15     IF NOT FOUND THEN
16         RAISE EXCEPTION 'Copy % is not currently on loan', p_barcode;
17     END IF;
18     IF p_return_date < v_loan.issue_date OR p_return_date > CURRENT_DATE THEN
19         RAISE EXCEPTION 'Invalid return date %', p_return_date;
20     END IF;
21
22     UPDATE loan SET return_date = p_return_date WHERE loan_id = v_loan.loan_id;
23
24     -- Fine assessment
25     SELECT mc.fine_per_day, mc.max_fine_per_loan INTO v_rate, v_cap
26     FROM member m JOIN member_category mc USING (category_id)
27     WHERE m.member_id = v_loan.member_id;
28
29     p_fine := fn_calculate_fine(v_loan.due_date, p_return_date, v_rate, v_cap);
30     IF p_fine > 0 THEN
31         INSERT INTO fine (loan_id, member_id, days_late, amount, assessed_on)
32         VALUES (v_loan.loan_id, v_loan.member_id,
33                 p_return_date - v_loan.due_date, p_fine, p_return_date);
34     END IF;
35
36     -- Hand the copy to the reservation queue (or back to the shelf)
37     p_held_for := fn_allocate_copy(v_loan.copy_id);
38 END $$;
```

A separate procedure `sp_pay_fine(fine_id, amount)` records full or part payments. It marks the fine PAID only when the balance reaches zero and rejects over-payment.

2.5 Handling of Reservations When All Copies Are Issued

A reservation is accepted *only* when no copy of the title is on the shelf; otherwise the member is told to borrow the available copy. Reservations form a **FIFO queue per title**, ordered by `reserved_at`. Figure 2 shows how the status of a copy and of a reservation change together.

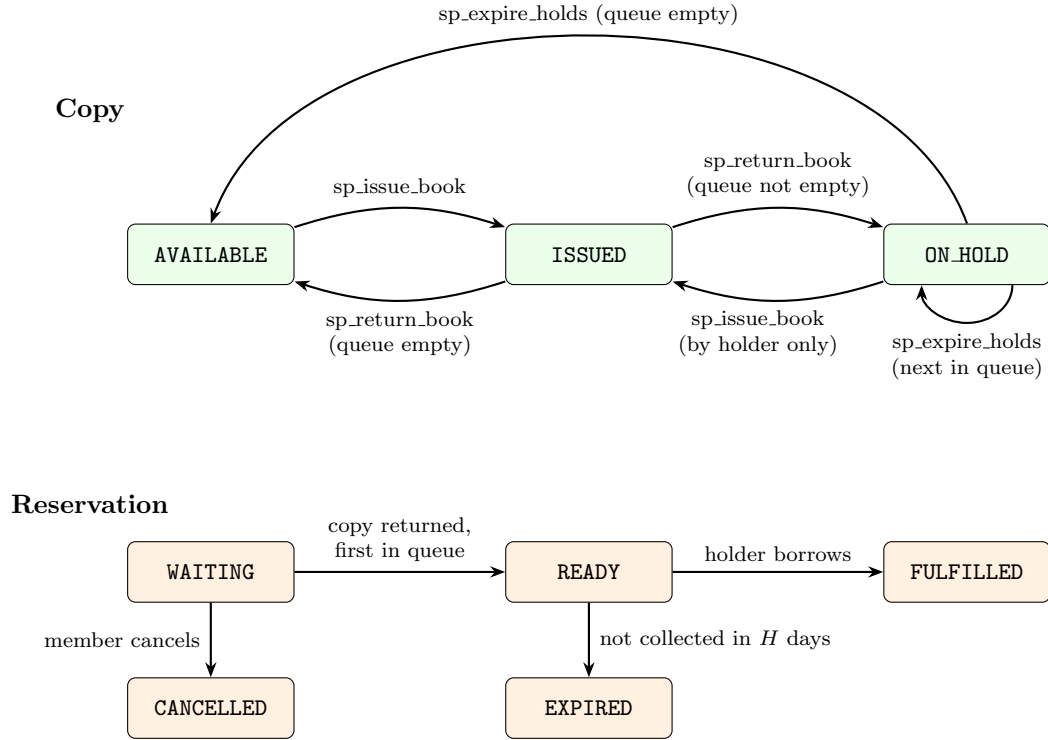


Figure 2: State transitions of a physical copy and of a reservation

The queue logic lives in one function, `fn_allocate_copy`, which is used by the return procedure, by the nightly hold-expiry job and when new copies are added:

Listing 4: Queue allocation and reservation procedures

```

1 CREATE OR REPLACE FUNCTION fn_allocate_copy(p_copy_id INT)
2 RETURNS INT LANGUAGE plpgsql AS $$
3 DECLARE
4     v_book_id INT;
5     v_resv     RECORD;
6 BEGIN
7     SELECT book_id INTO v_book_id FROM book_copy WHERE copy_id = p_copy_id FOR UPDATE;
8     -- Lock the title so a concurrent sp_reserve_book cannot slip in between
9     PERFORM 1 FROM book WHERE book_id = v_book_id FOR UPDATE;
10
11     SELECT reservation_id, member_id INTO v_resv
12     FROM reservation
13     WHERE book_id = v_book_id AND status = 'WAITING'
14     ORDER BY reserved_at, reservation_id
15     LIMIT 1
16     FOR UPDATE;
17
18     IF FOUND THEN
19         UPDATE reservation
20         SET status = 'READY', held_copy_id = p_copy_id,
21             hold_expires_on = CURRENT_DATE + 3 -- 3-day pickup window
22         WHERE reservation_id = v_resv.reservation_id;
23         UPDATE book_copy SET status = 'ON_HOLD' WHERE copy_id = p_copy_id;
24         RETURN v_resv.member_id;
25     END IF;
26
27     UPDATE book_copy SET status = 'AVAILABLE' WHERE copy_id = p_copy_id;
28     RETURN NULL;
29 END $$;
30
31 CREATE OR REPLACE PROCEDURE sp_reserve_book(p_member_id INT, p_book_id INT,
32                                             INOUT p_reservation_id INT DEFAULT NULL,
33                                             INOUT p_queue_position INT DEFAULT NULL)
34 LANGUAGE plpgsql AS $$

```

```

35 DECLARE
36     v_status VARCHAR;
37 BEGIN
38     SELECT status INTO v_status FROM member WHERE member_id = p_member_id;
39     IF NOT FOUND OR v_status <> 'ACTIVE' THEN
40         RAISE EXCEPTION 'Member % cannot place reservations', p_member_id;
41     END IF;
42
43     -- Serialise against fn_allocate_copy for the same title
44     PERFORM 1 FROM book WHERE book_id = p_book_id FOR UPDATE;
45     IF NOT FOUND THEN
46         RAISE EXCEPTION 'Book % does not exist', p_book_id;
47     END IF;
48
49     IF EXISTS (SELECT 1 FROM book_copy
50               WHERE book_id = p_book_id AND status = 'AVAILABLE') THEN
51         RAISE EXCEPTION 'A copy is on the shelf - issue it instead of reserving';
52     END IF;
53     IF EXISTS (SELECT 1 FROM loan l JOIN book_copy c USING (copy_id)
54               WHERE c.book_id = p_book_id AND l.member_id = p_member_id
55                     AND l.return_date IS NULL) THEN
56         RAISE EXCEPTION 'Member % already has this title on loan', p_member_id;
57     END IF;
58
59     BEGIN
60         INSERT INTO reservation (book_id, member_id)
61         VALUES (p_book_id, p_member_id)
62         RETURNING reservation_id INTO p_reservation_id;
63     EXCEPTION WHEN unique_violation THEN
64         RAISE EXCEPTION 'Member % already has a live reservation for book %',
65             p_member_id, p_book_id;
66     END;
67
68     SELECT COUNT(*) INTO p_queue_position
69     FROM reservation
70     WHERE book_id = p_book_id AND status = 'WAITING'
71     AND reservation_id <= p_reservation_id;
72 END $$;
73
74 -- Nightly job: expire holds not collected in time, pass copy onward
75 CREATE OR REPLACE PROCEDURE sp_expire_holds(INOUT p_expired INT DEFAULT 0)
76 LANGUAGE plpgsql AS $$
77 DECLARE
78     r RECORD;
79 BEGIN
80     p_expired := 0;
81     FOR r IN SELECT reservation_id, held_copy_id FROM reservation
82             WHERE status = 'READY' AND hold_expires_on < CURRENT_DATE
83     LOOP
84         -- lock order copy -> reservation, same as sp_issue_book (no deadlock)
85         PERFORM 1 FROM book_copy WHERE copy_id = r.held_copy_id FOR UPDATE;
86         UPDATE reservation SET status = 'EXPIRED'
87             WHERE reservation_id = r.reservation_id AND status = 'READY';
88         IF FOUND THEN
89             PERFORM fn_allocate_copy(r.held_copy_id);
90             p_expired := p_expired + 1;
91         END IF;
92     END LOOP;
93 END $$;

```

2.5.1 Concurrency Analysis

Two dangerous race conditions were identified and closed:

1. **Double issue / limit overrun.** Two counters issuing to the same member at the same moment could both read “2 of 3 books” and both succeed. `sp_issue_book` therefore locks the member row first (`FOR UPDATE OF m`), so the second transaction waits and then sees 3 open loans. The partial unique index `uq_loan_open_copy` is a second line of defence: even a buggy program cannot open two loans on one copy.

2. **Lost reservation.** If a member reserves a title at the same instant another member returns its last copy, the reservation could be stored while the copy goes to the shelf. Both `sp_reserve_book` and `fn_allocate_copy` lock the book row, so the two operations are serialised. Whichever runs second sees the other's committed result.

All procedures take locks in the same order (member → copy → book → reservation), which prevents deadlocks.

2.6 Overdue, Popular-Title and Member-wise Fine Queries

Listing 5: Q1 – Overdue books with the fine accrued so far

```

1 SELECT m.full_name AS member, mc.category_name AS category, b.title, c.barcode,
2       l.due_date, CURRENT_DATE - l.due_date AS days_overdue,
3       fn_calculate_fine(l.due_date, CURRENT_DATE,
4       mc.fine_per_day, mc.max_fine_per_loan) AS fine_so_far
5 FROM loan l
6 JOIN book_copy c ON c.copy_id = l.copy_id
7 JOIN book b ON b.book_id = c.book_id
8 JOIN member m ON m.member_id = l.member_id
9 JOIN member_category mc ON mc.category_id = m.category_id
10 WHERE l.return_date IS NULL
11 AND l.due_date < CURRENT_DATE
12 ORDER BY days_overdue DESC;

```

Table 3: Output of Q1 (executed on 05-Oct-2026)

Member	Category	Title	Barcode	Due date	Days late	Fine so far
Venkat Rao	STAFF	Operating System Concepts	LIB-0008	2026-09-26	9	9.00
Ananya Rao	STUDENT	Database System Concepts	LIB-0001	2026-09-29	6	12.00

Listing 6: Q2 – Most popular titles in the last 180 days

```

1 SELECT RANK() OVER (ORDER BY COUNT(l.loan_id) DESC) AS rnk,
2       b.title,
3       COUNT(l.loan_id) AS times_issued,
4       COUNT(DISTINCT l.member_id) AS borrowers,
5       (SELECT COUNT(*) FROM reservation r
6       WHERE r.book_id = b.book_id AND r.status = 'WAITING') AS waiting
7 FROM book b
8 JOIN book_copy c ON c.book_id = b.book_id
9 JOIN loan l ON l.copy_id = c.copy_id
10 WHERE l.issue_date >= CURRENT_DATE - 180
11 GROUP BY b.book_id, b.title
12 ORDER BY rnk, b.title
13 LIMIT 5;

```

Table 4: Output of Q2 – RANK() gives equal rank to ties

Rank	Title	Times issued	Distinct borrowers	Waiting
1	Database System Concepts	12	6	0
2	Clean Code	4	4	1
3	Fundamentals of Database Systems	3	3	0
3	Introduction to Algorithms	3	3	0
3	Operating System Concepts	3	2	0

Listing 7: Q3 – Member-wise fines: assessed, paid, outstanding and still accruing

```

1 WITH assessed AS (
2   SELECT member_id,
3          SUM(amount) AS total_assessed,
4          SUM(paid_amount) AS total_paid,

```

```

5      SUM(amount - paid_amount) FILTER (WHERE status = 'UNPAID') AS outstanding
6  FROM fine
7  GROUP BY member_id
8 ), accruing AS (
9  SELECT l.member_id,
10         SUM(fn_calculate_fine(l.due_date, CURRENT_DATE,
11                               mc.fine_per_day, mc.max_fine_per_loan)) AS accruing
12  FROM loan l
13  JOIN member m ON m.member_id = l.member_id
14  JOIN member_category mc ON mc.category_id = m.category_id
15  WHERE l.return_date IS NULL AND l.due_date < CURRENT_DATE
16  GROUP BY l.member_id
17 )
18 SELECT m.member_id, m.full_name,
19        COALESCE(a.total_assessed, 0)::NUMERIC(8,2) AS assessed,
20        COALESCE(a.total_paid, 0)::NUMERIC(8,2) AS paid,
21        COALESCE(a.outstanding, 0)::NUMERIC(8,2) AS outstanding,
22        COALESCE(x.accruing, 0)::NUMERIC(8,2) AS accruing,
23        (COALESCE(a.outstanding, 0) + COALESCE(x.accruing, 0))::NUMERIC(8,2) AS total_due
24 FROM member m
25 LEFT JOIN assessed a ON a.member_id = m.member_id
26 LEFT JOIN accruing x ON x.member_id = m.member_id
27 WHERE a.member_id IS NOT NULL OR x.member_id IS NOT NULL
28 ORDER BY total_due DESC, m.member_id;

```

Table 5: Output of Q3 (amounts in Rs.)

ID	Member	Assessed	Paid	Outstanding	Accruing	Total due
4	Kiran Kumar	52.00	0.00	52.00	0.00	52.00
1	Ananya Rao	0.00	0.00	0.00	12.00	12.00
2	Rahul Verma	10.00	0.00	10.00	0.00	10.00
8	Venkat Rao	0.00	0.00	0.00	9.00	9.00
5	Dr. S. Lakshmi	10.00	10.00	0.00	0.00	0.00
6	Mohammed Irfan	150.00	150.00	0.00	0.00	0.00
7	Sneha Reddy	12.00	12.00	0.00	0.00	0.00

Q3 separates *assessed* fines (books already returned late) from fines that are *still accruing* on books not yet returned. A member such as Ananya Rao owes nothing yet, but the desk can warn her that Rs. 12 is building up.

2.7 Testing and Validation

The sample data contains 8 members (6 students, 1 faculty, 1 staff), 8 titles, 13 copies, 24 historical/open loans, 4 fines and 3 reservations. All dates are relative to the current date, so the results are the same whenever the scripts are run. The test script `05_demo.sql` exercises every rule; Table 6 summarises the results.

Table 6: Test cases executed with 05_demo.sql

#	Scenario	Observed result	Pass
1	Student borrows an available copy (LIB-0003)	Loan created, due date = today + 14 days	✓
2	Student with 3 books tries a 4th	Borrowing limit reached: 3 of 3 books already issued	✓
3	Member with an overdue book tries to borrow	Member 1 has 1 overdue book(s); return them first	✓
4	Member owing Rs. 150 (> Rs. 100) tries to borrow	Unpaid fines Rs.150.00 exceed the limit of Rs.100.00	✓
5	Reservation while a copy is on the shelf	A copy is on the shelf - issue it instead of reserving	✓
6	Last copy of <i>Clean Code</i> returned, 2 members waiting	Copy set ON_HOLD for the first member (Priya)	✓
7	Another member tries to borrow the held copy	Copy LIB-0011 is on hold for another member	✓
8	Holder collects the copy	Loan created, reservation FULFILLED	✓
9	Book returned 26 days late by a student	Fine of Rs. 52.00 recorded	✓
10	Member pays Rs. 150 fine, then borrows	Balance 0.00, loan created	✓
11	Hold not collected within 3 days	Reservation EXPIRED, copy AVAILABLE	✓
12	Same member reserves the same title twice	Member 1 already has a live reservation for book 6	✓
13	Returning a copy that is not on loan	Copy LIB-0012 is not currently on loan	✓

2.8 Proposed Solution Strategy

Table 7: Summary of the design decisions

Strategy	Implementation Details
Data Modelling	Separate <code>book</code> (title) and <code>book_copy</code> (physical item); <code>loan</code> records the full issue/return cycle; normalized to BCNF.
Declarative Integrity	CHECK constraints on status and dates; partial unique indexes for “one open loan per copy” and “one live reservation per title per member”.
Business Rules	Category-driven limits, loan periods and fine rates in <code>member_category</code> ; enforced by <code>sp_issue_book</code> and <code>sp_return_book</code> .
Fair Reservations	Title-level FIFO queue, 3-day hold window, automatic hand-over through <code>fn_allocate_copy</code> and <code>sp_expire_holds</code> .
Concurrency Control	Row locks on member, copy and title rows taken in a fixed order to prevent double issue and deadlocks.
Reporting	Overdue, popularity (<code>RANK()</code>), member-wise fine (CTE) and demand-per-copy queries supported by partial indexes.

3 Sustainability Integration

3.1 Resource Efficiency and Data-Driven Acquisition

Every additional copy of a textbook costs money, paper and transport. The database can show exactly where extra copies are needed by comparing demand (loans plus people waiting) with supply (number of copies):

$$\text{Demand per copy} = \frac{\text{Loans in last 180 days} + \text{Waiting reservations}}{\text{Number of copies}}$$

Table 8: Demand per copy for the sample data (query Q4)

Title	Copies	Loans (180 days)	Waiting	Demand per copy
Clean Code	1	4	1	5.00
Database System Concepts	3	12	0	4.00
Introduction to Algorithms	2	3	0	1.50
Fundamentals of Database Systems	2	3	0	1.50
Operating System Concepts	2	3	0	1.50

Clean Code has the highest demand per copy and a waiting queue, so it is the best candidate for a second copy. Titles with low demand per copy need no further purchase. Buying according to measured demand avoids surplus copies that would sit unused on shelves, which supports **SDG 12**. Replacing paper issue registers and printed overdue notices with database records and digital reminders saves paper as well.

3.2 Impact on Society and Environment

- **Social:** The FIFO queue, the hold window and the fine cap treat every member equally and predictably. Students who cannot buy expensive textbooks still get timely access (**SDG 4, SDG 10**).
- **Environmental:** Shared use of each physical copy and demand-based purchasing reduce paper consumption and waste.
- **Economic:** Accurate fine records and fewer lost books reduce financial losses for the institution. The fine-blocking rule encourages members to return books on time.

4 Conclusion and Future Directions

4.1 Solution Summary

The case study modelled a library as ten normalized relations, with a clear separation between bibliographic titles and physical copies. Borrowing limits, due dates and fine rules are stored as category data and enforced by PL/pgSQL stored procedures. Reservations are handled through a fair, title-level FIFO queue with automatic hand-over and hold expiry. Row-level locking and partial unique indexes keep the data consistent even when several counters work at once. The overdue, popularity and member-wise fine queries give librarians the information they need, and all thirteen test scenarios produced the expected results.

4.2 Future Scope

Future work includes **RFID/barcode self-checkout kiosks** that call the same stored procedures, automatic **e-mail/SMS reminders** before the due date and when a hold becomes

ready, and **online fine payment**. The circulation history could also drive a **book recommendation system** and a **dashboard** for the library committee. For very large libraries, the **loan** table can be range-partitioned by year to keep queries fast.

Bibliography

- [1] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2016.
- [3] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, Vol. 13, No. 6, pp. 377–387, 1970.
- [4] P. P. Chen, “The Entity-Relationship Model – Toward a Unified View of Data,” *ACM Transactions on Database Systems*, Vol. 1, No. 1, pp. 9–36, 1976.
- [5] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*: Chapter 43 (PL/pgSQL) and Section 13.3 (Explicit Locking), <https://www.postgresql.org/docs/16/>.
- [6] United Nations, *Transforming Our World: The 2030 Agenda for Sustainable Development*, A/RES/70/1, 2015.